

Take-Home Technical Assignment

Junior Backend Developer

Portfolio Optimizer API Challenge

Field	Details
Role	Junior Backend Developer
Time Limit	12 hrs
Difficulty	Intermediate
Reference Tool	https://finominal.com/portfolio-optimizer/US

Overview

Welcome! This assignment evaluates your ability to design and implement a clean, well-structured REST API. You will be replicating the core optimization engine behind Finominal's Portfolio Optimizer tool.

You will be provided with the historical return data for 5 funds. Your task is to build an API that accepts these returns and a chosen optimization strategy, then returns the optimized portfolio weights — mirroring the output of the live tool.

You can use the live tool at the link above to validate your outputs against a reference implementation.

What You Will Receive

Along with this document, you will be provided:

- An Excel file containing historical return data for 5 funds
 - Historical factor return data for the Momentum, Value, and Size factors
 - Ticker symbols, security names, and dividend yield data for each fund
 - Access to the live [Portfolio Optimizer](#) tool to compare and validate your outputs
-

Your Task

Build a REST API with an endpoint that accepts:

- A list of securities (tickers) with their return data
- An optimization strategy name
- Optional constraints (e.g., min/max weights per security and portfolio-level constraints such as Min CAGR, Volatility Range, Max Drawdown, and Min Dividend Yield)

The API must return:

- Optimized portfolio weights for each security (the Allocation Changes section)
- (Bonus) Factor betas for both the current and optimized portfolio

Optimization Strategies to Implement

Implement the following strategies in the order listed. Each builds on core portfolio math concepts:

#	Strategy	Description	Status
1	Equal Weights	Assign equal allocation to all securities. Simple baseline — sum of weights must equal 100%.	Required
2	Risk Parity	Allocate weights such that each security contributes equally to total portfolio risk (based on volatility).	Required
3	Minimize Drawdown	Find the portfolio allocation that minimizes the maximum historical drawdown over the given time period.	Required
4	Minimize Volatility	Find the allocation that produces the lowest portfolio volatility (standard deviation of returns).	Required
5	Maximize Sharpe Ratio	Find the allocation that maximizes the Sharpe Ratio (risk-adjusted return). You may assume a risk-free rate of 0% or use a standard value.	Required
6	Optimize Factor Exposure	Maximize or minimize exposure to one or more equity risk factors (Value, Momentum, Size). Factor returns will be provided in the same excel file.	Bonus

Bonus: Implement strategy #6 (Optimize Factor Exposure) along with Factor Betas in the API response. This is not required but will be looked upon very favourably.

Factor Exposure Calculation

For the bonus Factor Exposure strategy, calculate factor betas by first constructing the portfolio return series from the selected funds and their weights.

Then regress the portfolio's historical returns against the provided factor return series: Momentum, Value, and Size.

Use the common date range available between the portfolio returns and factor returns.

Portfolio Return = $\alpha + \beta_1(\text{Momentum}) + \beta_2(\text{Value}) + \beta_3(\text{Size}) + \text{error}$

Run this calculation for both the original portfolio and the optimized portfolio, and return factor betas for both.

Factor Exposure Validation

For the Factor Exposure bonus, candidates are not expected to match the live tool exactly. The assignment uses only three factors, while the live tool uses a broader internal model.

The goal is to show a clear approach. For the Momentum test case, the optimized portfolio should have higher Momentum exposure than the original portfolio, while still respecting all weight and constraint rules.

Expected API Response Format

Your API must return a JSON response. At minimum, it must contain the Allocation Changes section. The Bonus is to also return Factor Betas.

Required: Allocation Changes

```
{  "optimization_strategy": "minimize_volatility",  "allocation_changes": [    {      "ticker": "AGG",      "security_name": "iShares Core US Aggregate Bond ETF",      "current_weight": 50.00,      "optimized_weight": 37.65,      "change": -12.35    },    {      "ticker": "DPG",      "security_name": "Duff & Phelps Utility and Infrastructure Fund Inc",      "current_weight": 50.00,      "optimized_weight": 62.35,      "change": 12.35    }  ] }
```

Bonus: Factor Betas

If you implement Factor Exposure optimization, also return factor betas for both the current portfolio and the optimized portfolio:

```
{  "factor_betas": {    "current_portfolio": {      "value": 0.09,      "momentum": 0.10,      "size": 0.12    },    "optimized_portfolio": {      "value": 0.14,      "momentum": 0.18,      "size": 0.08    }  }
```

```
}  
}  
}
```

You can cross-reference these values using the live tool's Comparison tab, which shows Factor Betas for both the current and optimized portfolio.

How to Validate Your Output

Use the live Portfolio Optimizer tool to verify your results:

1. Go to <https://finominal.com/portfolio-optimizer/US>
2. Enter the 5 provided funds with equal weights (50/50 to start)
3. Select an optimization strategy from the dropdown
4. Click Run Optimization
5. Compare the Allocation Changes table in the Review Results tab with your API output
6. (Bonus) Compare the Factor Betas section in the Comparison tab

Your weights should match the tool's output closely. Small differences are acceptable.

Required Test Scenarios

Candidates must validate their API against the live Portfolio Optimizer tool using the following six test cases. The first five cases are required. Case 6 applies to candidates attempting the Factor Exposure bonus. For each case, submit the API response and a screenshot from the live tool for comparison. Optimized weights should closely match the reference tool output. Small floating-point differences below 0.1% are acceptable.

#	Scenario	Input Portfolio	Strategy	Constraints
1	Basic equal-weight sanity check	IEFA: 25%, SPY: 75%	Equal Weights	None
2	Risk-based allocation across different asset classes	VEA: 25%, AGG: 75%	Risk Parity	None
3	Lowest-risk portfolio construction	SPY: 60%, AGG: 30%, GLD: 10%	Minimize Volatility	None
4	Risk-adjusted return optimization	IEFA: 20%, GLD: 20%, AGG: 20%, VEA: 20%, SPY: 20%	Maximize Sharpe Ratio	None
5	Portfolio-level and security-level constraint handling	IEFA: 20%, GLD: 20%, AGG: 20%, VEA: 20%, SPY: 20%	Maximize Sharpe Ratio	Min Dividend Yield: 2.50%; each security min: 5%, max: 40%
6	Momentum factor exposure	IEFA: 20%, GLD: 20%, AGG: 20%, VEA: 20%, SPY: 20%	Optimize Factor Exposure	Maximize Momentum

Acceptance checks for these scenarios

- Optimized weights must sum to 100%.
- No optimized weight should be negative.
- Case 5 must respect both the Min Dividend Yield constraint and the security-level min/max weight limits.
- The response should include ticker, security name, current weight, optimized weight, and allocation change.
- If a constraint is infeasible, the API should return a clear validation error instead of silently producing invalid weights.

-
- Case 6 should maximize Momentum exposure where feasible and include factor betas for both the current and optimized portfolio.

Technical Requirements

Language & Framework

- You may use any Python language framework (Django, FastAPI, Flask etc)
- The API must be runnable locally — provide clear setup instructions

Code Quality

- Clean, readable, and well-commented code
- Proper error handling (invalid tickers, missing data, unsupported strategy)
- A README.md explaining how to set up and run the project
- Unit tests are a bonus but not required

Constraints

- Weights must always sum to 100%
- No individual weight may be negative (no short selling)
- If min/max weight constraints are passed in the request, they must be respected
- If portfolio-level constraints are passed in the request, they must be respected where feasible

Submission Requirements

Once complete, send the following within 12 hours:

Deliverable	Details
GitHub Repo Link	Public repository with all source code and a README.md with setup instructions
Screenshots	Screenshots showing your API running and returning correct output for at least 3 optimization strategies
Loom Video	A short screen recording (3-5 mins) walking through your code, your approach, and demonstrating the API working

Your Loom video is your chance to explain your thinking. We care about how you approach problems, not just the final output. Narrate your decisions, trade-offs, and any shortcuts you took due to time constraints.

Evaluation Criteria

You will be evaluated on:

Criterion	What We Look For	Weight
Correctness	Weights match the reference tool's output	40%

Code Quality	Readability, structure, error handling, comments	30%
Communication (Loom)	Clarity of explanation, reasoning, trade-offs	20%
Bonus: Factor Exposure + Betas	Correct factor beta output matching the tool	+10% Bonus

Tips & Hints

- Start with Equal Weights — it requires no optimization and will help you get the API structure right first
- Risk Parity and Minimize Volatility are standard problems solvable with `scipy.optimize` or similar libraries
- You don't need to build a frontend — a clean REST API with Postman/curl-testable endpoints is sufficient
- Use the live tool actively to sanity-check your results as you build
- If you get stuck on a strategy, implement what you can and explain your approach in the Loom video — partial credit is given

Good luck! We look forward to seeing your work.

If you have any questions, please reach out to kaushik@finominal.com.